

METODOLOGÍA DE LA PROGRAMACIÓN

Guión de prácticas de laboratorio

Profesores: Jorge Puente, Ramiro Varela, Alfredo Alguero, Rubén Muñiz, Pablo Suarez-Otero

TEMA 2. Programación Orientada a Objetos. POLIMORFISMO

Material para dejar en el Campus Virtual: Fichero zip con proyectos completos, o bien partes de proyectos que hay que completar, con sus correspondientes ejemplos de prueba. En algunos casos se incluyen diagramas de clases del proyecto final.

- Jerarquía de bicicletas: **Bicycle**, **MountainBike**, **RoadBike** (Proyecto 03_BicycleInheritancePolimorphism)
- Jerarquía básica de trabajadores: clases **CommissionEmployee** y **BasePlusCommissionEmployee** con un ejemplo de uso con polimorfismo (Proyecto Fig_10_1_Polimorphism)
- Interface **Driveable** y clase **Bicycle** (Proyecto 041_BicycleInterfazDriveable)
- Ejemplo de clase de downcasting erróneo (Proyecto DowncastErroneo)
- Jerarquía de trabajadores con una clase abstracta **Employee**, y varios niveles de herencia (Proyecto Fig_10_04_09_PayrollSystem)
- Interface **Payable** y clases **Invoice**, **Employee** y **SalariedEmployee** (Proyecto Fig_10_11_15_PayableInterface)
- Clase **FiguraGeometrica** (Proyecto Figuras)
- Interface **FiguraGrafica** (Proyecto FigurasGUI)
- Jerarquía de clases de la versión JEROQUEST 2.0 (NO ESTÁ DE MOMENTO)

PRACTICA 2.1. Uso de jerarquías de clases simples con polimorfismo

Objetivo: Entender bien los ejemplos de polimorfismo vistos en las clases de teoría con las jerarquías de bicicletas y trabajadores

Ejercicio 2.1.1. Uso de polimorfismo con las clases **CommissionEmployee** y **BasePlusComissionEmployee** definidas por herencia

- a) Accede al proyecto Fig_10_1_Polymorphism y ejecuta el ejemplo de prueba que incluye.
- b) Crea un vector de trabajadores que incluya trabajadores de las dos clases **CommissionEmployee** y **BasePlusComissionEmployee**, y muestra los datos del trabajador que más gana.
- c) Muestra por consola todos los trabajadores, pero ordenados por su salario de menor a mayor.
- d) Compara los algoritmos diseñados en b) y en c) con los que se diseñaron en las prácticas del tema 1 (Ejercicios 1.2.1 b) y c)) y escribe un comentario sobre las ventajas que aporta el uso del polimorfismo en este caso.

Ejercicio 2.1.2. Uso de polimorfismo en las clases de la jerarquía de bicicletas

- e) Accede al proyecto 03_BicycleInheritancePolimorphism, ejecuta el ejemplo de prueba y añade alguna bicicleta más al vector de bicicletas.
- f) Haz un recorrido del vector de bicicletas aplicando el método **getSuspension** a todas las bicicletas y observa el error que se produce.
- g) Diseña un algoritmo para recorrer el vector de bicicletas, de modo que para las bicicletas de montaña se muestre solamente el tipo de suspensión, para las de carretera el valor del campo **tireWidth** y para los objetos de la clase **Bicycle** el estado completo (con **printStates** o utilizando **toString**).

PRACTICA 2.2. Ejemplos con la jerarquía completa de trabajadores

Objetivo: Entender la funcionalidad de una jerarquía no trivial, con clases abstractas, varios niveles de herencia y polimorfismo

Ejercicio 2.2.1. Utilizar la jerarquía completa de trabajadores

- Accede al proyecto Fig_10_04_09_PayrollSystem y ejecuta el programa de prueba.
- Modifica la acción del esquema condicional

```
if ( currentEmployee instanceof BasePlusCommissionEmployee )
{ . . . }
```

de forma que no sea preciso definir una nueva variable de la clase **BasePlusCommissionEmployee**
- Incluye en el programa de prueba los algoritmos diseñados en el ejercicio 2.1.1, b) y c), para calcular el trabajador que más gana y ordenar los trabajadores según su salario de menor a mayor. Comprueba que funcionan perfectamente en este ejemplo que incluye una jerarquía con un número mayor de clases.

Ejercicio 2.2.2. Diseñar algún algoritmo nuevo que utilice la jerarquía completa de trabajadores

- Implementa mediante un método estático de la clase **PayrollSystemTest** un algoritmo para incrementar el valor del campo **commissionRate** de los trabajadores que hay en un vector de trabajadores y que tienen este dato, de acuerdo con la especificación siguiente:

```
/**
 * @param employees un vector de trabajadores
 * @param inc lo que hay que incrementar commissionRate
 * @param aQuien entero que indica a quienes hay que cambiar
 * el valor: 0 a todos los que tienen este dato, 1 solo a los
 * de la clase CommissionEmployee, 2 solo a los de la clase
 * BasePlusCommissionEmployee
 */
public static void cambiarComissionRate( Employee[] employees,
                                         double inc, int aQuien){...}
```

PRACTICA 2.3. Definición y uso de interfaces

Objetivo: Entender la funcionalidad de las interfaces en Java y apreciar las diferencias y similitudes de las relaciones **extends** e **implements**

Ejercicio 2.3.1. Uso de la interface **Driveable** en la clase **Bicycle**

- Accede al proyecto 041_BicycleInterface y utiliza el ejemplo de prueba.
- Modifica el proyecto de a) de modo que la clase **Bicycle** se declare como abstracta y que tenga las subclases **MountainBike** y **RoadBike** definidas de forma análoga a como están definidas en el ejercicio 2.1.2 (por supuesto teniendo en cuenta que ahora existe la interfaz **Driveable**). Modifica **bike1** y **bike2** para que se le asigne una nueva **MountainBike** y **bike3** para que se le asigne una nueva **RoadBike**.
- Cambia la jerarquía de modo que sean **MountainBike** y **RoadBike** las que implementen la interfaz **Driveable** en lugar de **Bicycle**. Comprueba si se produce algún error en el programa de prueba y en caso afirmativo corrígelo.

Ejercicio 2.3.2. Uso de la interface **Payable** en las clases **Employee** e **Invoice**.

- Accede al proyecto Fig_10_11_15_PayableInterface y ejecuta el ejemplo de prueba. Observa que a las variables cuyo tipo es la interface **Payable** se les pueden aplicar métodos de la clase **Object**.
- Incluye otra clase, por ejemplo **HourlyEmployee** para representar a trabajadores que trabajan por horas, cuyo salario se calcula como **wage * hours**, para las primeras 40 horas trabajadas, mientras que las siguientes horas son extraordinarias y se computan como 1,5 horas cada una en el salario.
- Crea un vector con facturas y con trabajadores de todas las clases y haz un recorrido del vector para mostrar lo que hay que pagar por cada trabajador y por cada factura. En el caso de los trabajadores por horas indica también el número de horas trabajadas.

PRACTICA 2.4. Ejemplo más complejo, con clases abstractas, interfaces y representaciones gráficas.

Objetivo: Entender un ejemplo no trivial con relaciones **extends** e **implements**.

Ejercicio 2.4.1. En la jerarquía de la práctica 1.5 del tema 1, define una nueva clase **Triangulo** y trátala polimórficamente junto con la clase **CirculoConCentro**.

- Accede al proyecto Figuras. El programa de prueba no funcionará hasta que completes los apartados b) y c).
- Añade la clase **Triangulo** (los datos de esta clase son 3 puntos en el plano, tiene un constructor a partir de las 6 coordenadas de sus vértices (x1,y1,x2,y2,x3,y3) , un método para calcular la superficie y otro para el perímetro).
- Define una clase abstracta **FiguraGeometrica** de la que hereden tanto **Triangulo** como **Circulo**. Esta clase en principio no tendrá ningún atributo, y como métodos abstractos: **double area()** y **double perimetro()**. Esta clase nos servirá para manejar polimórficamente las figuras. Para ello definiremos en el programa principal un array de objetos **FiguraGeometrica** y guardaremos en él varios círculos, círculos con centro y triángulos, para luego recorrerlo y mostrar cada uno de ellos en modo texto.

Ejercicio 2.4.2. Se trata ahora de diseñar algoritmos para las clases **CirculoConCentro** y **Triangulo** y de implementarlos en métodos definidos en la jerarquía anterior.

- Diseña el método estático en la clase **FiguraGeometrica**:

```
/**
 * Calcula el área total de un array de figuras geométricas
 * @param figs array de figuras geométricas
 * @return el área total de todas las figuras
 */
static public double areaTotal(FiguraGeometrica[] figs)
```

Modifica el programa principal para que haga uso de este método para calcular el área total de las figuras creadas en los apartados anteriores.

- Crea un vector de círculos y triángulos y diseña algoritmos de recorrido y/o búsqueda en el vector para:
 - Comprobar si es mayor la superficie total de los triángulos o la de los círculos.

- b. Ordenar las figuras del vector por superficie de menor a mayor.

Ejercicio 2.4.3. Se trata ahora de añadir una interface para representar de forma gráfica objetos de las clases **Triangulo** y **CirculoConCentro**.

- a) Accede al prototipo FigurasGUI y observa lo que contiene. Incorpora la jerarquía completa de **FiguraGeometrica** del ejercicio anterior.
- b) Define la interfaz **FiguraGrafica**. Esta interfaz ofrecerá tres métodos:

```
/**
 * Pinta la figura y su etiqueta en la ventana que se le
 * especifica utilizando el objeto Graphics que se le
 * proporciona
 * @param g objeto Graphics sobre el que pintar la figura
 */
public void pinta(Graphics g);

/**
 * Fija la etiqueta a mostrar junto con la figura
 * @param nuevaEtiqueta etiqueta a asociar a la figura
 */
public void setEtiqueta(String nuevaEtiqueta);

/**
 * Devuelve la etiqueta asociada a la figura
 */
public String getEtiqueta();
```

- c) Implementa esta interfaz en las clases **CirculoConCentro** y **Triangulo**. (en **Circulo** no la definimos ya que los círculos no tienen ubicación en el plano). Para pintar círculos con centro tenemos el código en la práctica de **CirculosGUI** del tema anterior. Para pintar los triángulos podéis usar el siguiente método:

```
public void pinta(Graphics g)
{
    // creamos un array con las Xs y otro con las Ys
    int [] xs = {v1.getX(), v2.getX(), v3.getX()};
    int [] ys = {v1.getY(), v2.getY(), v3.getY()};

    // creamos un objeto polígono con esos 3 puntos
    Polygon p = new Polygon(xs, ys, 3);

    // dibujamos el polígono
    g.drawPolygon(p);

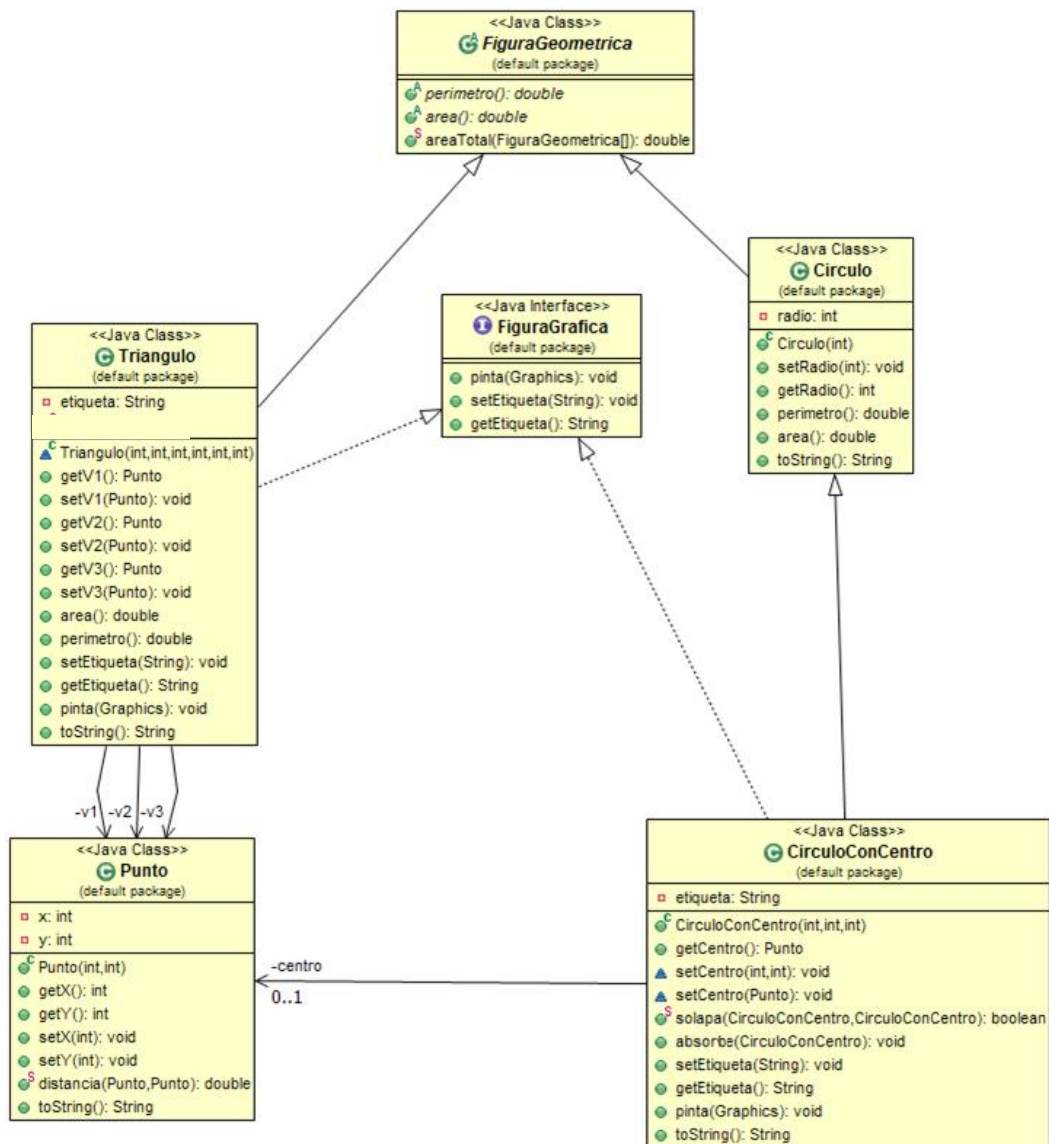
    // dibujamos la etiqueta
    int x = (int)((v1.getX() + v2.getX() + v3.getX())/3);
    int y = (int)((v1.getY() + v2.getY() + v3.getY())/3);
    g.drawString(this.getEtiqueta(), x - 3, y + 4);
}
```

d) Modifica la clase **MiPanel** de modo que:

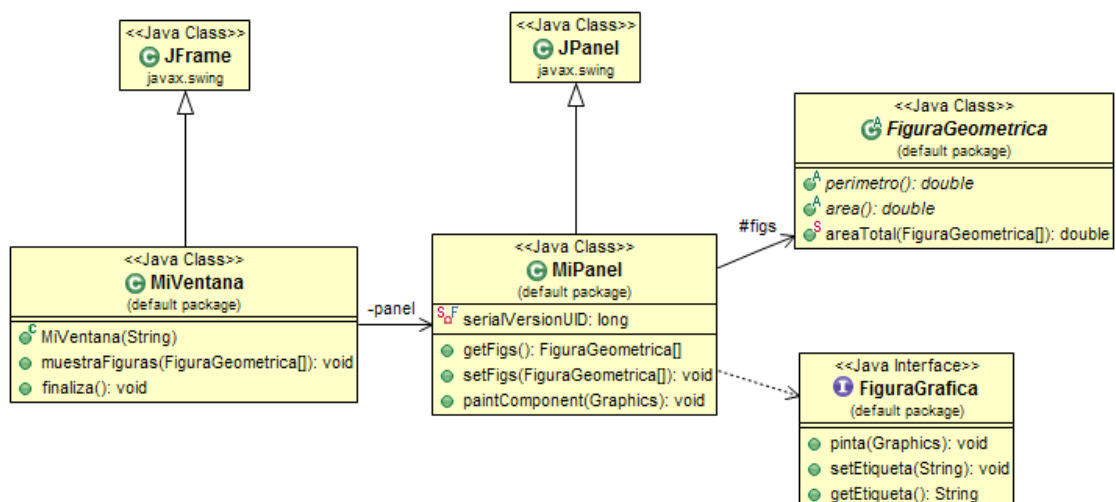
- El atributo **figs** sea un array de objetos **FiguraGeometrica**, en lugar de un array de objetos **CirculoConCentro**.
- Modifica el método **MiPanel.getFigs()** y **MiPanel.setFigs()** para que trabajen con un array de **FiguraGeometrica**.
- Modifica también el método **MiPanel.paintComponent()** para que identifique que figuras son también **FiguraGrafica** y que mediante un casting invoque a su método **pinta()**.
- Finalmente, en la clase **MiVentana**, cambia el nombre del método **muestraCirculos()** de modo que pase a llamarse **muestraFiguras()** y que reciba un array de objetos **FiguraGeometrica**.

e) Modifica el programa principal para que además de crear las figuras les asigne a cada una su etiqueta (por ejemplo para círculos: **CN**, y para triángulos **TM**, siendo **N** y **M** números enteros del 0 en adelante). Utiliza un objeto **MiVentana** y su método **muestraFiguras()** para visualizar (junto con sus etiquetas) todas aquellas figuras geométricas que además sean figuras gráficas.

El diagrama de clases resultante de los ejercicios anteriores es el que se muestra en la figura siguiente. Observa que en primer lugar vemos como **MiVentana** y **MiPanel** heredan de las correspondientes clases **JFrame** y **JPanel** de Java. Además podemos ver como la clase **MiPanel** 'tiene un' array de **FiguraGeometrica** para referenciar las figuras a mostrar. Y además 'hace uso' (o depende) de la interface **FiguraGrafica** a la hora de pintar las figuras.



El diagrama de clases relativo al interfaz gráfico sería el siguiente:



PRACTICA 2.5. JEROQUEST 2.0

Objetivo: Aplicar todos nuestros conocimientos de Herencia y Polimorfismo en una aplicación no trivial: el JEROQUEST

(SE PUBLICARÁ LA VERSIÓN COMPLETA PRÓXIMAMENTE)

DISTRIBUCIÓN TEMPORAL

Sesión 10: PRACTICA 2.1.

Sesión 11. PRACTICA 2.2.

Sesión 12. PRACTICA 2.3

Sesión 13. PRÁCTICA 2.4. Ejercicios 2.4.1 y 2.4.2.

Sesión 14. PRÁCTICA 2.4. Ejercicio 2.4.3.

Sesión 15. PRÁCTICA 2. 5.

Sesión 16. PRÁCTICA 2.5.

Sesión 17. PRÁCTICA 2.5.

Sesión 18. PRÁCTICA 2.5.